

# Manual Operativo: Entrenamiento en Google Colab (GPU A100) y Próximos Pasos

Esta guía detalla el procedimiento exacto paso a paso para que tú o cualquier investigador/desarrollador del equipo pueda entrenar el modelo denso, evaluar las métricas científicas, generar las tablas LaTeX para la publicación y clonar el sistema en cualquier entorno.

## 1. Estructura de los Datasets de Entrenamiento

Los datasets ubicados en `../backend/data/` fueron generados y auditados matemáticamente con **Cero Fuga de Datos (*Document-Level Out-of-Distribution*)**:

| Archivo                               | Cantidad de Tripletas | % del Corpus | Rol en la Investigación                                                                                                   |
|---------------------------------------|-----------------------|--------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>train_triplets.json</code>      | 1,918                 | 70%          | Ajuste supervisado de los 560M parámetros con pérdida <i>Multiple Negatives Ranking Loss (MNRL)</i> .                     |
| <code>val_triplets.json</code>        | 1,024                 | 15%          | Evaluación periódica del ranking ( <code>TripletEvaluator</code> ) durante el entrenamiento para evitar sobreajuste.      |
| <code>test_triplets_blind.json</code> | 694                   | 15%          | Evaluación ciega sobre 10 Guías de Práctica Clínica jamás vistas durante el entrenamiento ( <i>Out-of-Distribution</i> ). |
| <code>ft_dataset.json</code>          | 3,636                 | 100%         | Dataset global consolidado de todas las tripletas generadas con <i>Hard Negatives</i> .                                   |

Estructura de cada Triplet:

```
{
  "id": "triplet_00078",
  "query": "¿Cuál es el esquema de tratamiento farmacológico normado por el MSP para fenilcetonuria?",
  "pos": "Diagnóstico y tratamiento nutricional del paciente pediátrico con fenilcetonuria...",
  "neg": "Diagnóstico, tratamiento y seguimiento del paciente con Enfermedad de Gaucher...",
  "guia_fuente": "2013_guia_de_fenilcetonuria",
  "seccion": "Tratamiento Farmacológico y Nutricional",
  "ano_publicacion": 2013,
  "especialidad": "Genética y Hematología",
  "tipo_negativo": "Hard Negative Intra-Especialidad"
}
```

---

## 2. Paso a Paso: Entrenamiento en Google Colab Pro (GPU NVIDIA A100)

### Paso 1: Abrir el Notebook en Google Colab

1. Ve a [Google Colab](#).
2. Haz clic en **Subir (Upload)** y selecciona el archivo  
`../backend/ingestion/colab_fine_tuning.ipynb`.

### Paso 2: Seleccionar la GPU A100

1. En el menú superior de Colab: **Entorno de ejecución > Cambiar tipo de entorno de ejecución**.
2. Selecciona **A100 GPU** (o en su defecto V100/T4 si estás en Colab estándar).
3. Haz clic en **Guardar**.

### Paso 3: Cargar los Datos

1. En el panel izquierdo de Colab, abre la pestaña de **Archivos** (icono de carpeta).
2. Arrastra y suelta los dos archivos desde tu computadora:
  - `backend/data/train_triplets.json`
  - `backend/data/val_triplets.json`

### Paso 4: Ejecutar el Entrenamiento

1. Presiona `Ctrl + F9` o ve a **Entorno de ejecución > Ejecutar todas**.
2. El script detectará la GPU A100, configurará el tamaño de lote grande (\$B=32\$) y la ventana de \$1024\$ tokens, y entrenará el modelo durante 3 épocas.
3. **Tiempo estimado:** ~8 a 10 minutos.

### Paso 5: Descargar los Artefactos Generados

Al finalizar la última celda, el notebook generará y descargará automáticamente:

1. `ateneo-bge-m3-ecuador.zip`: Archivo comprimido con los pesos del modelo optimizado.
2. `grafico_convergencia_paper.png`: Gráfico formal de pérdida y exactitud a **300 DPI**, listo para insertar en el artículo científico.

---

## 3. Despliegue del Modelo Entrenado en tu Computadora

1. Descomprime el archivo descargado `ateneo-bge-m3-ecuador.zip`.
2. Coloca la carpeta resultante en: `backend/data/ateneo-bge-m3-ecuador/`
3. ¡Listo! El sistema `../backend/config.py` detectará los nuevos pesos y conmutará automáticamente todas las búsquedas hacia el modelo afinado con las 45 Guías del MSP.

---

## 4. Próximos Pasos: Ingesta y Generación de Tablas LaTeX para el Paper

Tienes 2 opciones para ejecutar la ingesta masiva de PDFs y los benchmarks científicos:

## Opción A: Aceleración Élite en GPU NVIDIA A100 + Google Drive (Recomendada - Cero Esperas)

Para evitar lentitudes en el navegador y desconexiones por inactividad:

1. Sube el archivo `../ateneo_colab_bundle.zip` a tu **Google Drive** en la carpeta **Mi unidad > Proyectos > Ateneo**.
  2. Abre en **Google Colab** el notebook maestro:  
`../backend/ingestion/colab_ingesta_benchmark_a100.ipynb`.
  3. Selecciona **GPU A100** y presiona **Ctrl + F9 (Ejecutar todas)**.
  4. El notebook:
    - Monta Google Drive y transfiere el archivo en 2 segundos a 1 Gbps+.
    - Extrae tablas en Markdown de los PDFs y genera los embeddings con **BF16** en segundos.
    - Ejecuta el Benchmark Cuantitativo (**Tabla I**) y el Estudio de Ablación (**Tabla II**).
    - **Guarda una copia de seguridad directa en Proyectos/Ateneo de tu Google Drive** y además te descarga: `chroma_db.zip`, `tabla_resultados_paper.tex` y `tabla_ablacion_paper.tex`.
  5. Descomprime `chroma_db.zip` en tu carpeta local `backend/data/chroma_db/`.
- 

## Opción B: Ejecución en Máquina Local (CPU)

Si prefieres correrlo en tu máquina local:

### 1. Ingesta y Vectorización Local

```
cd backend
py ingestion/run_ingestion.py
```

### 2. Tabla I: Benchmark de Rendimiento IR (In vs Out of Distribution)

```
cd backend
py tests/run_metrics.py
```

- **Qué hace:** Evalúa el sistema frente a 25 casos clínicos reales anotados, discriminando el rendimiento en guías de entrenamiento (*In-Distribution*) vs guías ciegas (*Out-of-Distribution*).
- **Salida:** Emite el código LaTeX listo para pegar en el paper con las métricas  $\text{Hit@1}$ ,  $\text{Hit@3}$ ,  $\text{Hit@5}$ ,  $\text{MRR@5}$ ,  $\text{NDCG@5}$  y latencias  $P_{50}/P_{95}$ .

### 3. Tabla II: Estudio de Ablación de Componentes

```
cd backend
py tests/run_ablation_study.py
```

- **Qué hace:** Demuestra el impacto científico individual de cada componente (Búsqueda Léxica pura, Embeddings genéricos, Embeddings Fine-Tuned y RAG Híbrido RRF).
- **Salida:** Emite el código LaTeX de la Tabla de Ablación para la sección de Resultados del artículo.

---

## 5. Cómo Clonar, Migrar y Correr en Otra Máquina (Flujo MLOps con Google Drive)

Los archivos binarios gigantes (los pesos de 2.27 GB del modelo y la base vectorial binaria de ChromaDB) **NO se suben a GitHub** por estándar de la industria (límite de 100 MB y prevención de repositorios pesados). Todo el respaldo centralizado reside en tu **Google Drive (Mi unidad > Proyectos > Ateneo)**.

Para levantar el proyecto en cualquier otra computadora:

```
# 1. Clonar el repositorio desde GitHub (código fuente, frontend, backend, PDFs y docs)
git clone https://github.com/JorgeDoicela/clinical_rag.git
cd clinical_rag

# 2. Descargar los 2 artefactos binarios desde Google Drive (Carpeta: Proyectos > Ateneo):
#   A. 'Modelo entrenado.zip' -> Descomprimir en: backend/data/ateneo-bge-m3-ecuador/
#   B. 'chroma_db.zip'         -> Descomprimir en: backend/data/chroma_db/

# 3. Iniciar el Sistema Completo (Backend + Frontend) con Docker:
docker compose up -d --build
```

### Alternativa sin Docker (Ejecución Local):

- **Backend:** `cd backend && py main.py` (corre en `http://localhost:8000`)
- **Frontend:** `cd frontend && npm install && npm run dev` (corre en `http://localhost:5173`)

---

## 6. Registro de Compatibilidad de Modelos Fine-Tuned y Base Vectorial

- **Notas de Versión en ateneo-bge-m3-ecuador:** Cuando el modelo es exportado desde Google Colab u otros entornos con versiones recientes de `sentence-transformers`, sus archivos de configuración (`sentence_bert_config.json`, `modules.json` y `1_Pooling/config.json`) deben mantener el esquema estándar compatible con la versión de producción (`sentence-transformers==3.3.1`). La plantilla oficial y compatible está comiteada en `master` (`c827982`).
- **Persistencia HNSW en ChromaDB 0.6.3:** En el despliegue con Docker, los metadatos de los segmentos vectoriales (`index_metadata.pickle`) se instancian como objetos tipados `PersistentData` fijando explícitamente `dimensionality=1024` y `space='cosine'`, previniendo errores de carga y evitando re-ingestas redundantes de los 45 PDFs en tiempo de inferencia.
- **Rúbrica Psicométrica Calibrada:** La evaluación con Gemini aplica una escala de 5 niveles con anclaje en evidencia (\$0.0\$ a \$10.0\$) que penaliza rigurosamente omisiones totales (\$0.0\text{ pts}\$ ante "no

sé" o en blanco) y desglosa las brechas en los 4 ejes clínicos (Diagnóstico, Tratamiento, Prevención, Seguimiento).